



Introduction to Kserve

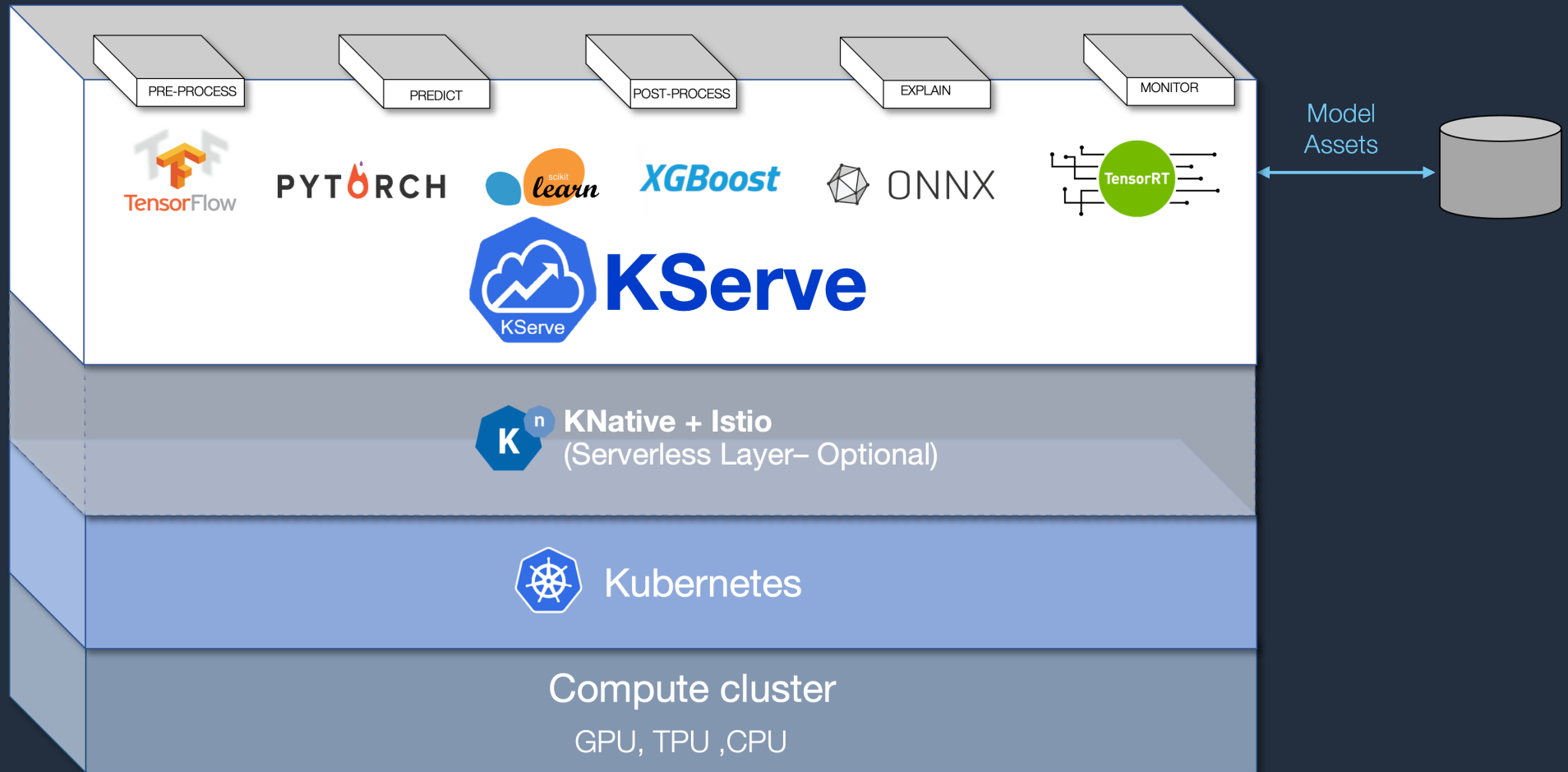
Keita Watanabe

Sr. Solutions Architect, AI/ML Frameworks
AWS

Agenda

- KServe Overview
- KServe Components
 - Inference Service
 - Predictor
 - AutoScaling with Knative Pod Autoscaler (KPA)
- ML inference with KServe Examples

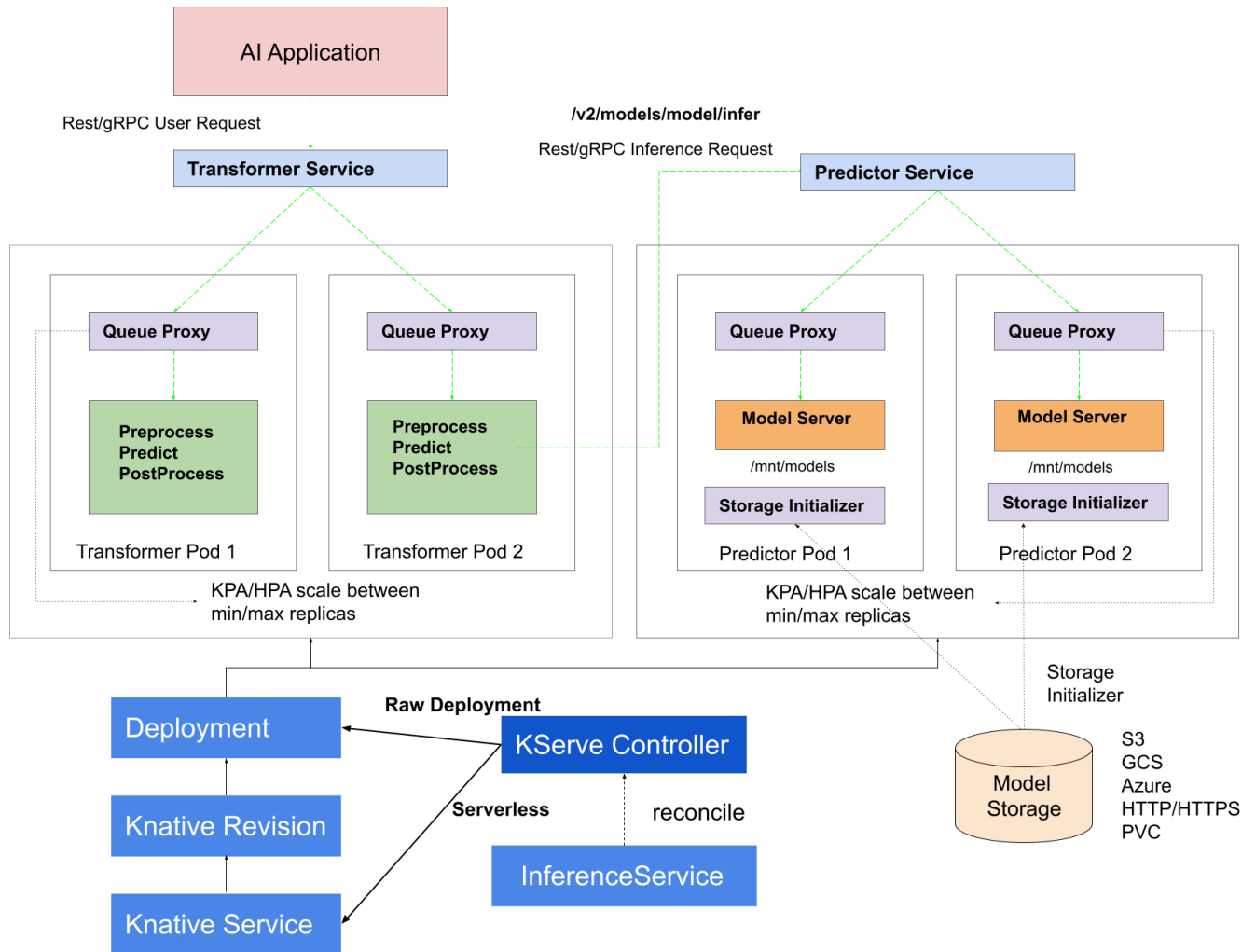
KServe



KServe Features

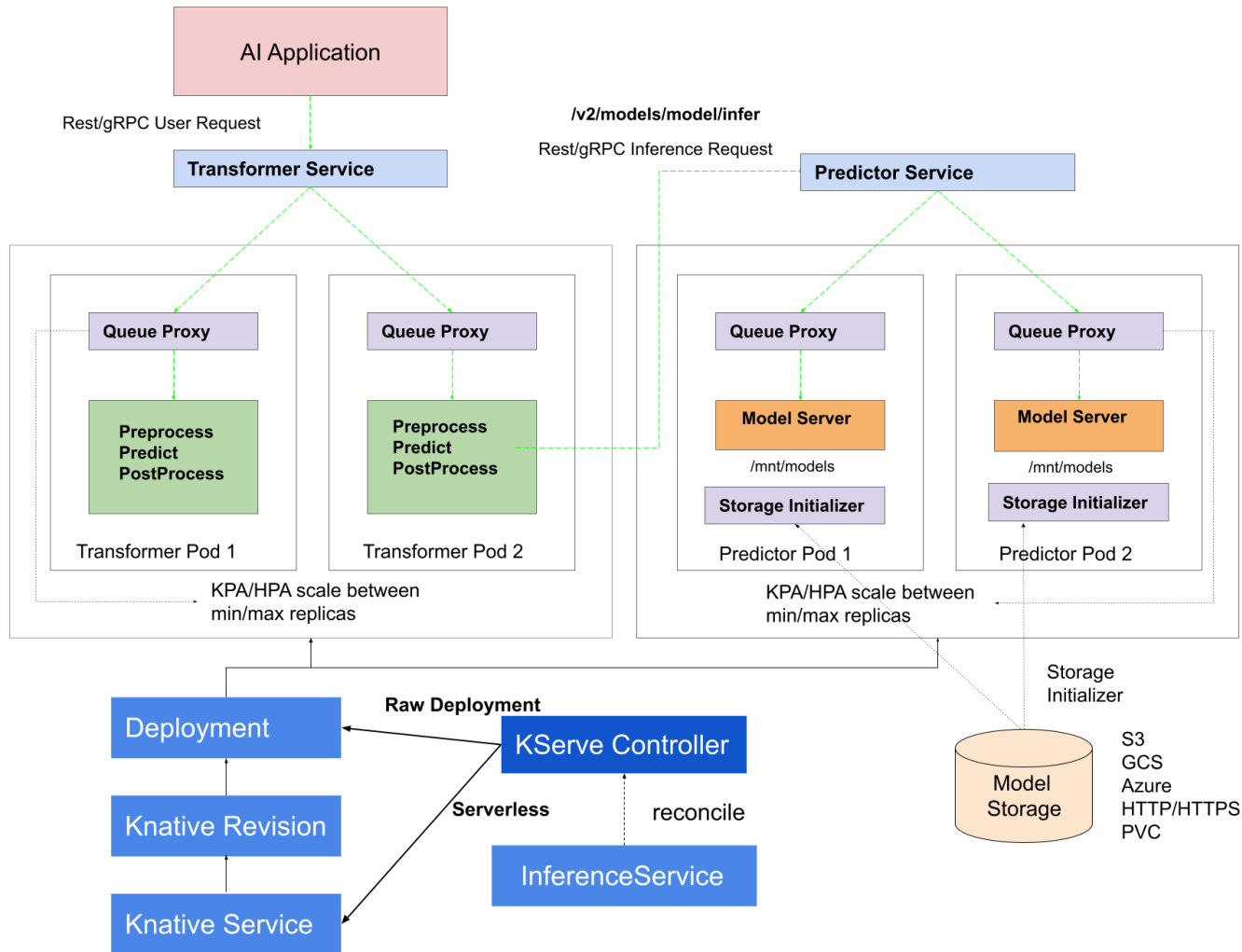
- Scale to and from Zero
- Request based Autoscaling
- Batching
- Request/Response logging
- Traffic management
- Security with AuthN/AuthZ
- Distributed Tracing
- Out-of-the-box metrics

KServe Control Plane



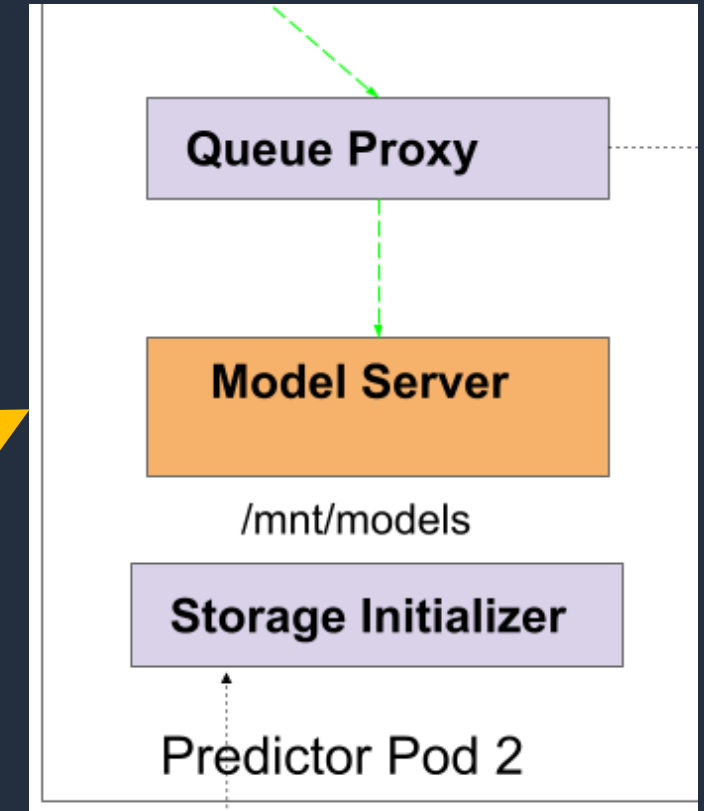
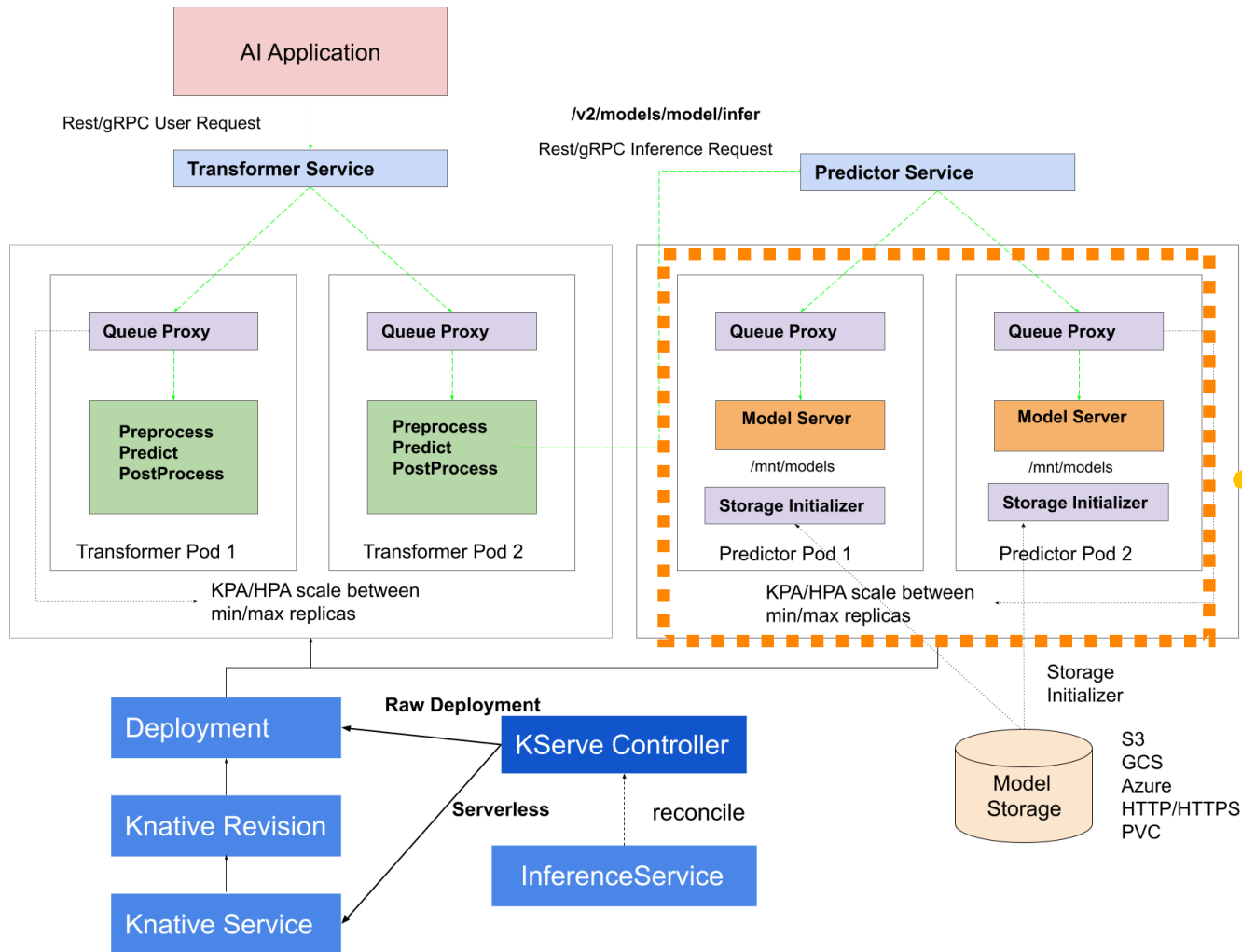
- Responsible for reconciling the **InferenceService** custom resources.
- It creates the **Knative serverless deployment** for predictor, transformer to enable **autoscaling** based on incoming request workload including scaling down to zero when no traffic is received.

KServe Control Plane

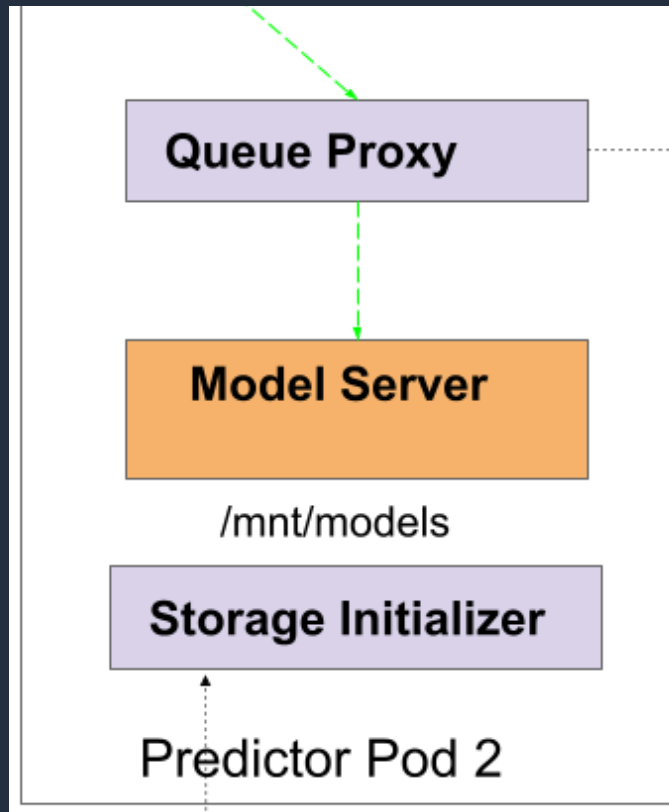


- Responsible for reconciling the **InferenceService** custom resources.
- It creates the **Knative serverless deployment** for predictor, transformer to enable **autoscaling** based on incoming request workload including scaling down to zero when no traffic is received.

Predictor



Predictor

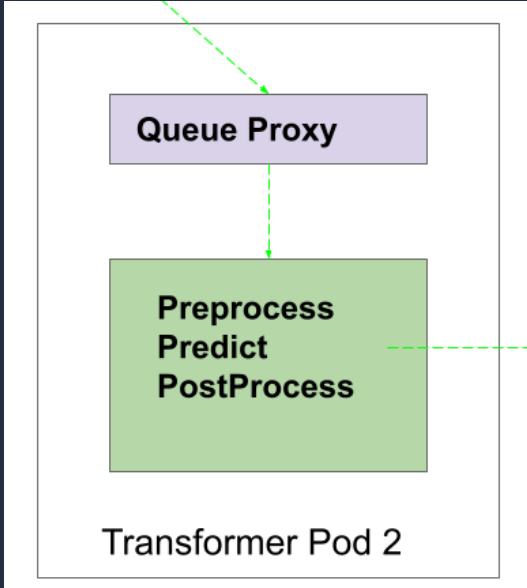
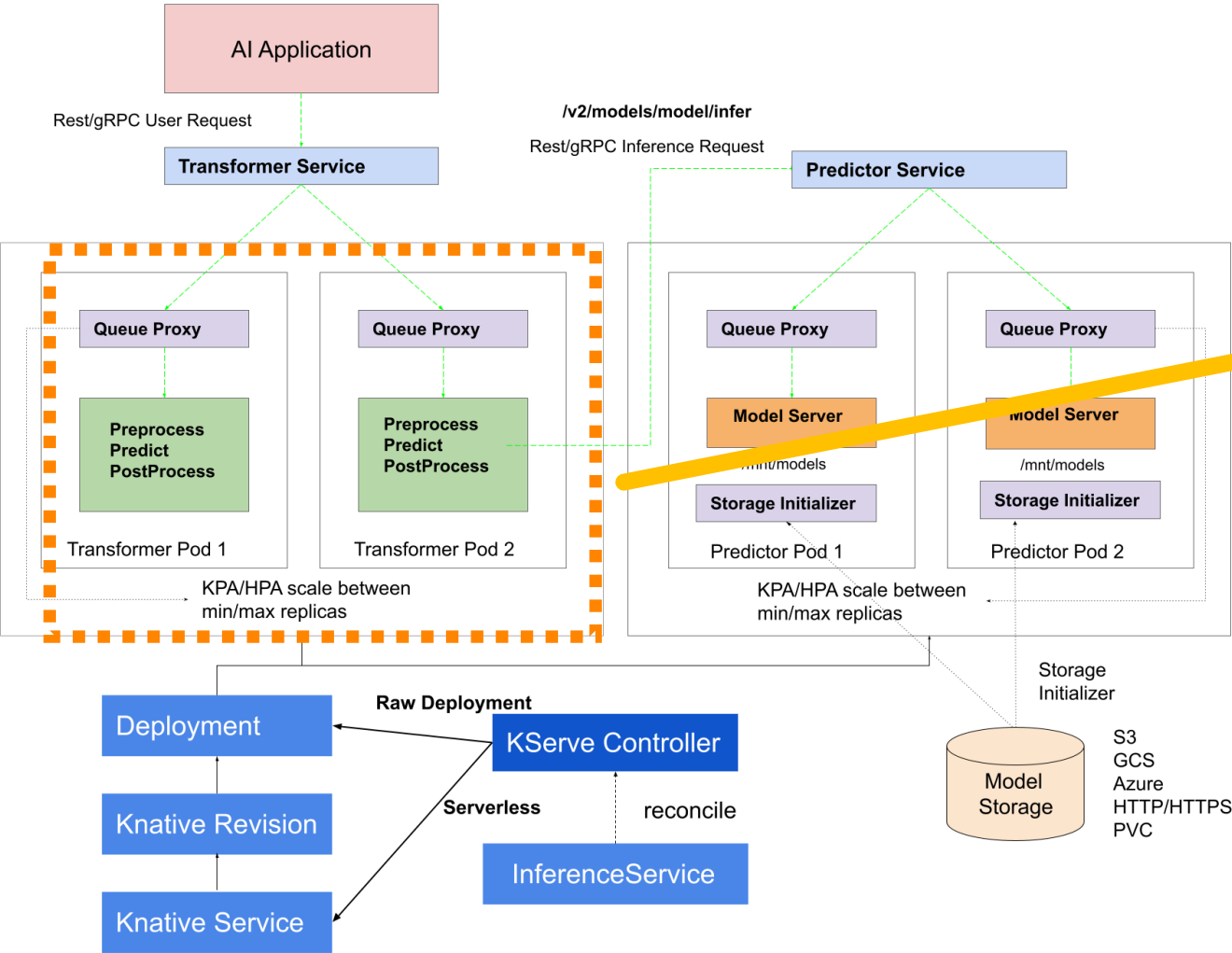


Queue Proxy measures and limit concurrency to the user's application

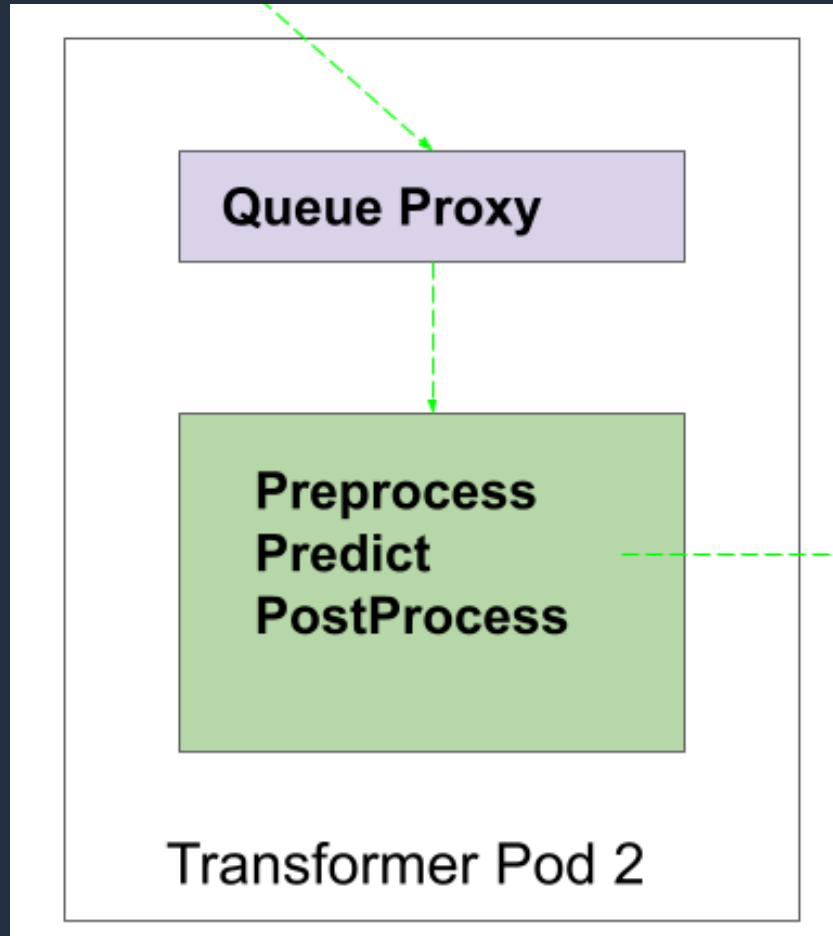
Model Server deploys, manages, and serves machine learning models

Storage Initializer retrieves and prepares machine learning models from various storage backends like Amazon S3

Transformer



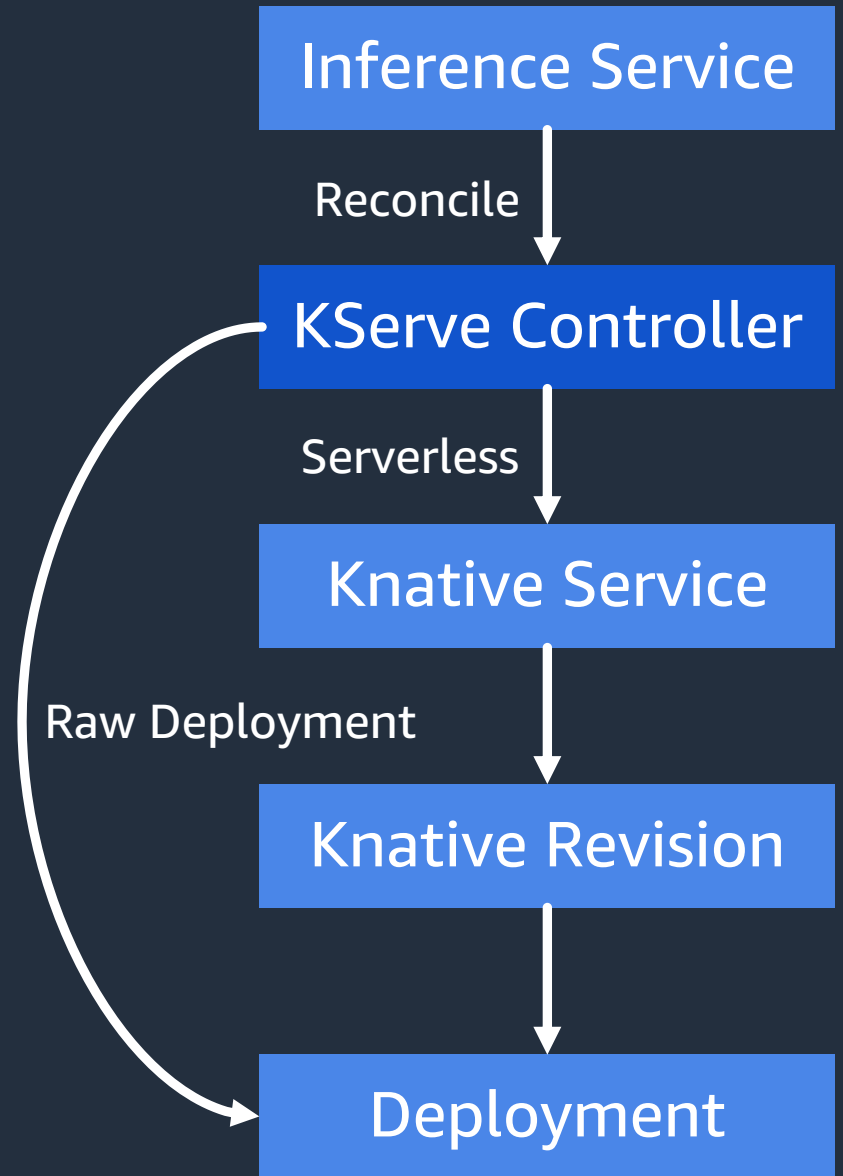
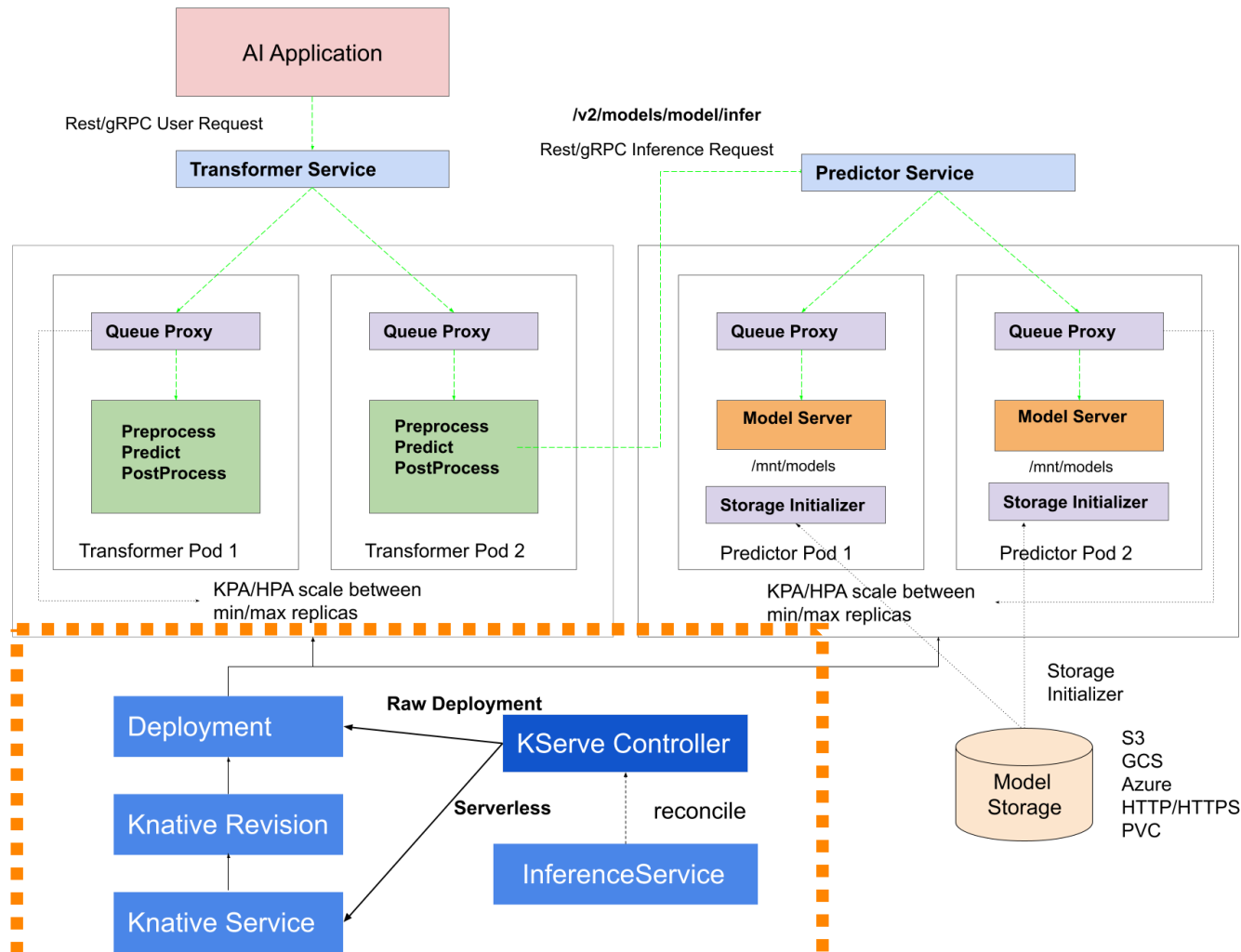
Transformer



Queue Proxy measures and limits concurrency to the user's application.

Model Server preprocesses input data and postprocesses output predictions, enabling seamless integration of custom logic or data transformations with the deployed machine learning models for improved model serving and inference.

KServe Control Plane



The diagram illustrates the KServe architecture, which is designed for serving AI models. It is divided into two main sections: the **Transformer Service** and the **Predictor Service**.

Transformer Service: This service handles **Rest/gRPC User Request**. It consists of two pods, **Transformer Pod 1** and **Transformer Pod 2**. Each pod contains a **Queue Proxy** and a **Preprocess Predict PostProcess** block. The **Queue Proxy** receives requests and passes them to the **Preprocess Predict PostProcess** block. The **Preprocess Predict PostProcess** block in **Transformer Pod 2** is connected to the **Model Server** in **Predictor Pod 1**.

Predictor Service: This service handles **Rest/gRPC Inference Request**. It consists of two pods, **Predictor Pod 1** and **Predictor Pod 2**. Each pod contains a **Queue Proxy** and a **Model Server**. The **Queue Proxy** in **Predictor Pod 1** is connected to the **Model Server**. The **Model Server** in **Predictor Pod 1** is connected to the **Storage Initializer** in **Predictor Pod 2**. The **Storage Initializer** in **Predictor Pod 2** is connected to the **Model Storage**.

KServe Controller: The **KServe Controller** manages the deployment of the services. It interacts with the **Deployment**, **Knative Revision**, and **Knative Service** components. The **KServe Controller** is also connected to the **Storage Initializer** in **Predictor Pod 2** and the **Model Storage**.

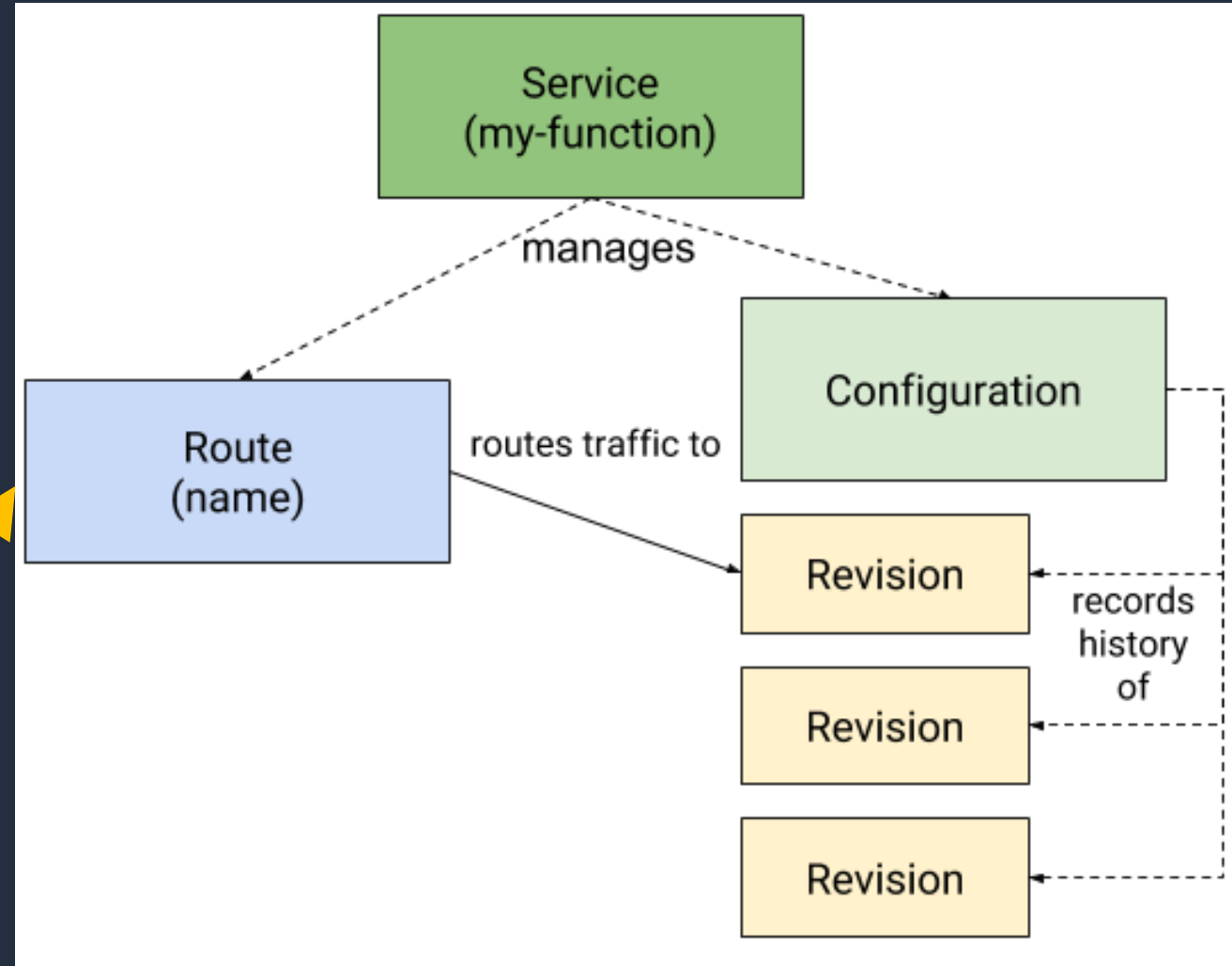
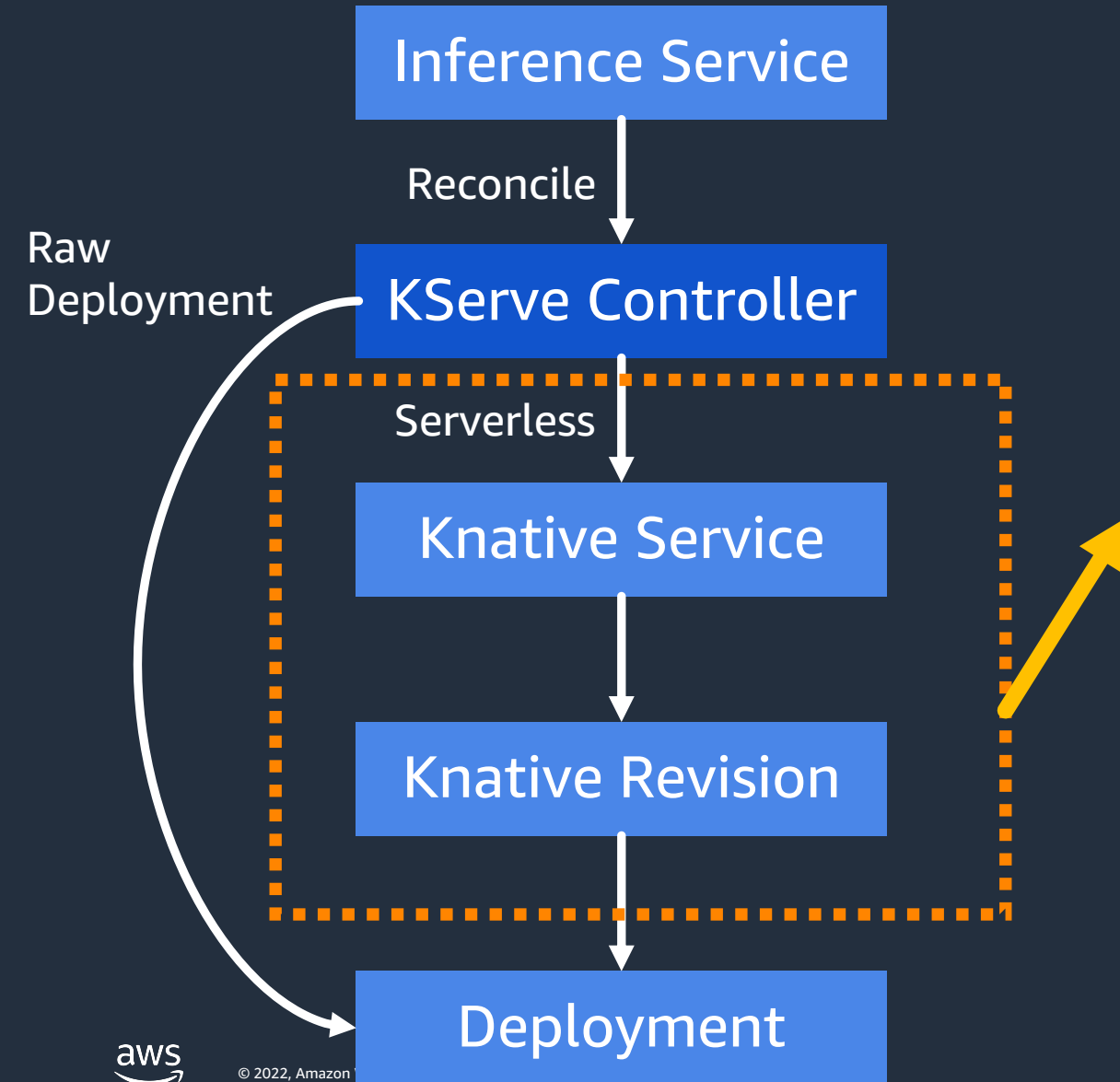
Model Storage: The **Model Storage** is a storage component that stores the model files. It is connected to the **Storage Initializer** in **Predictor Pod 2** and the **KServe Controller**. The **Model Storage** supports various storage providers: **S3**, **GCS**, **Azure**, **HTTP/HTTPS**, and **PVC**.

Deployment and Scaling: The **Deployment** component is responsible for scaling the services. It uses **KPA/HPA scale between min/max replicas** to manage the number of replicas. The **Knative Revision** and **Knative Service** components are also involved in the deployment and scaling process.

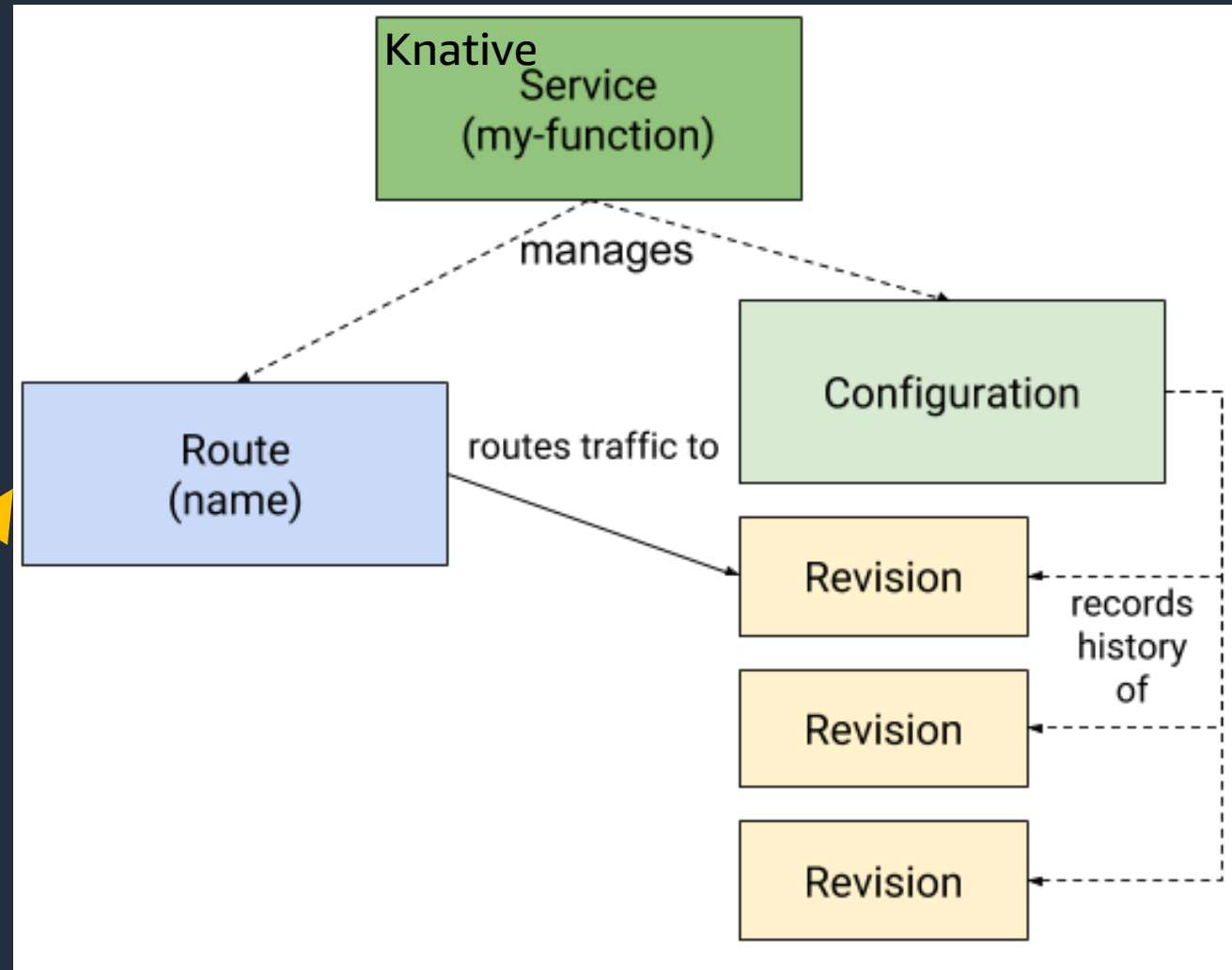
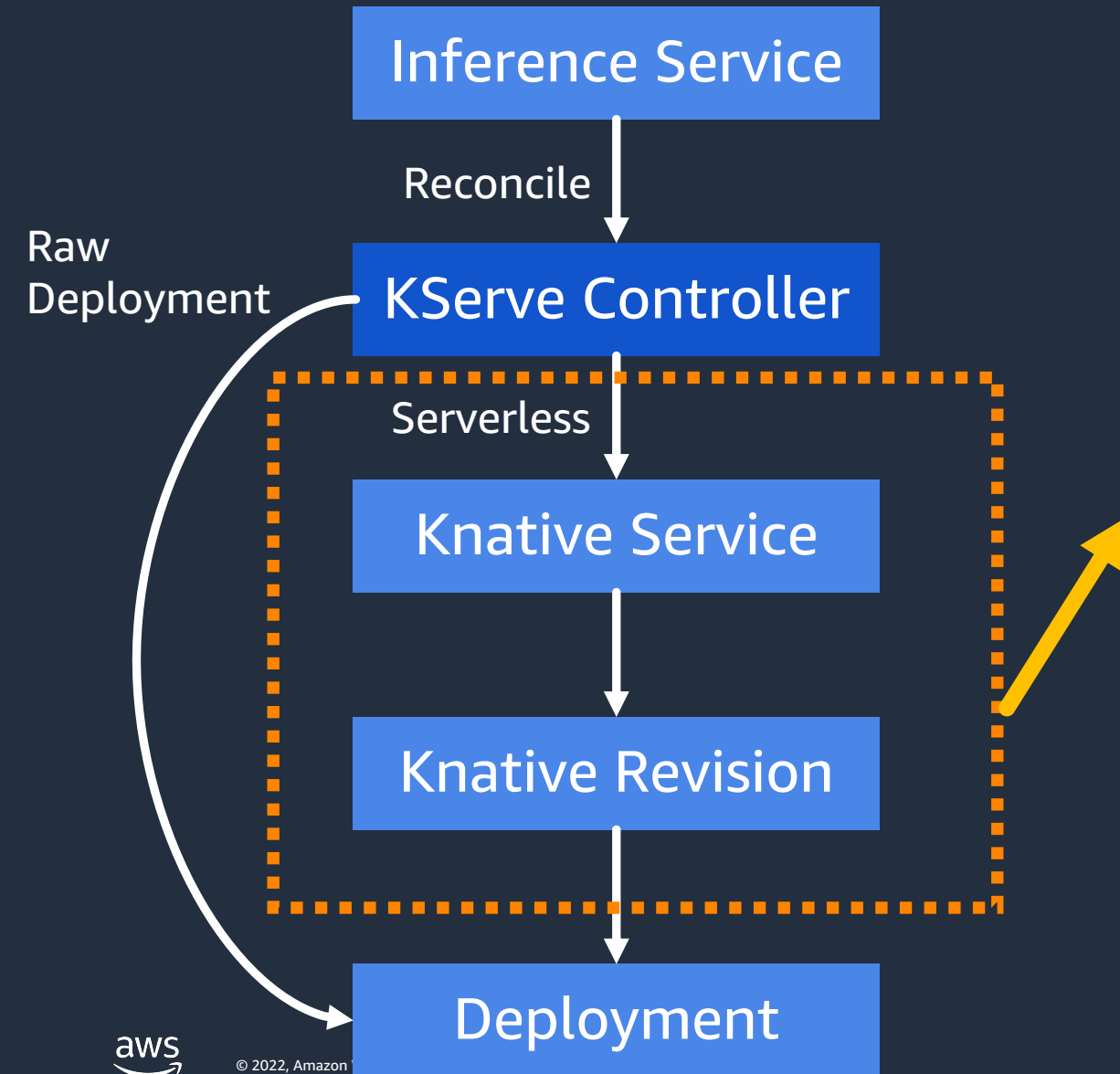
Reconcile: The **reconcile** process is used to ensure that the state of the system matches the desired state. It is managed by the **KServe Controller** and involves the **InferenceService** component.



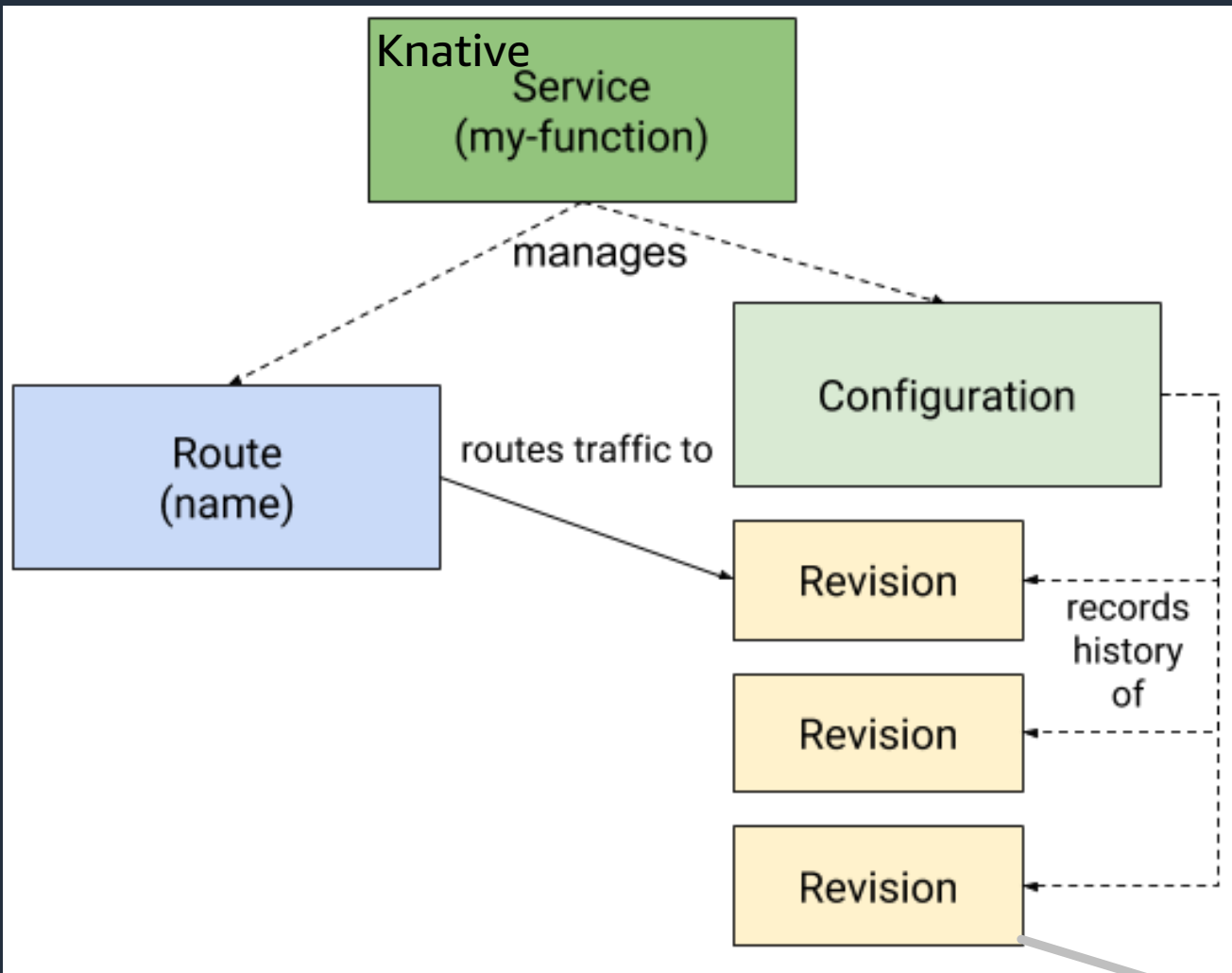
Knative Components



Knative Serving



Primary Knative Serving Resources



Knative Service resource automatically manages the whole lifecycle of your workload.

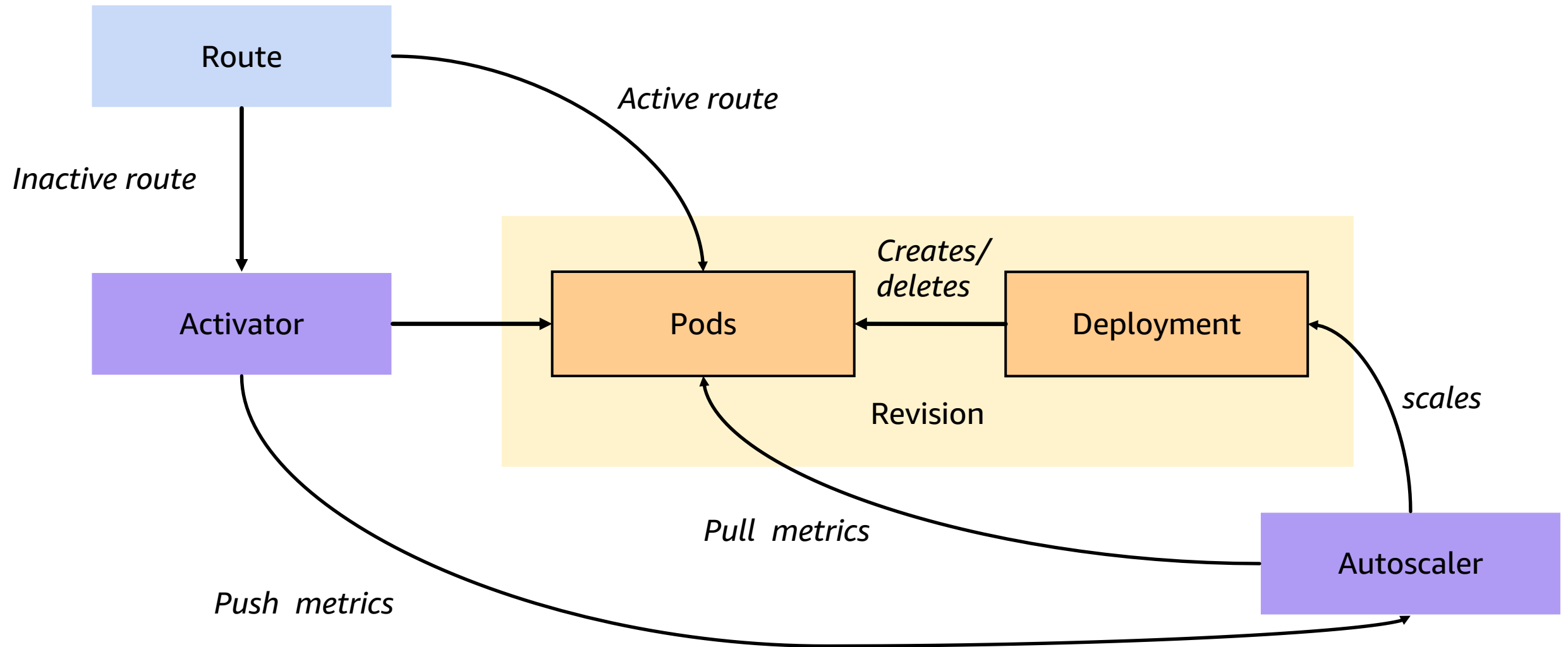
Routes maps a network endpoint to one or more revisions.

Configuration maintains the desired state for your deployment.

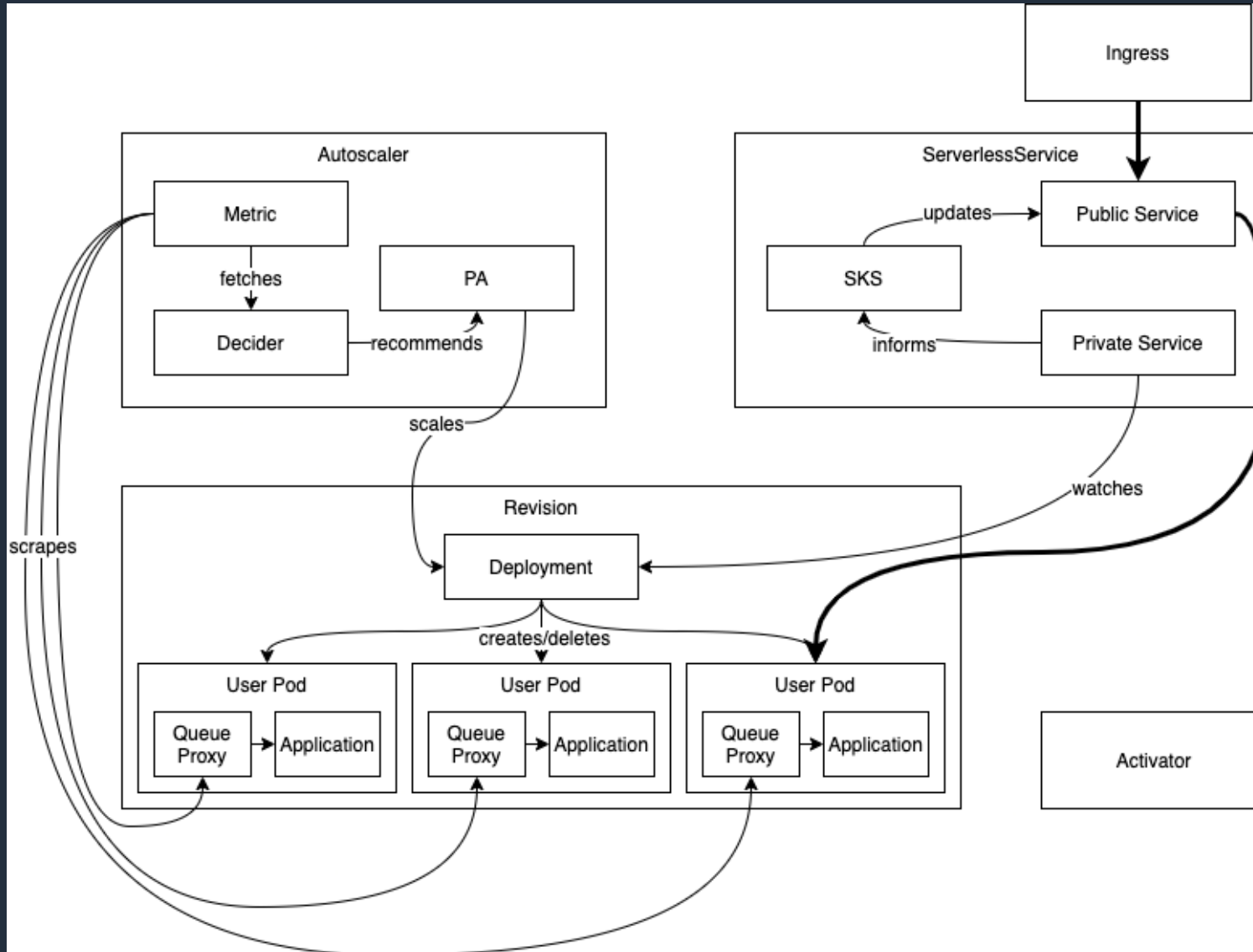
Revision is a point-in-time snapshot of the code and configuration for each modification made to the workload.

Deployment

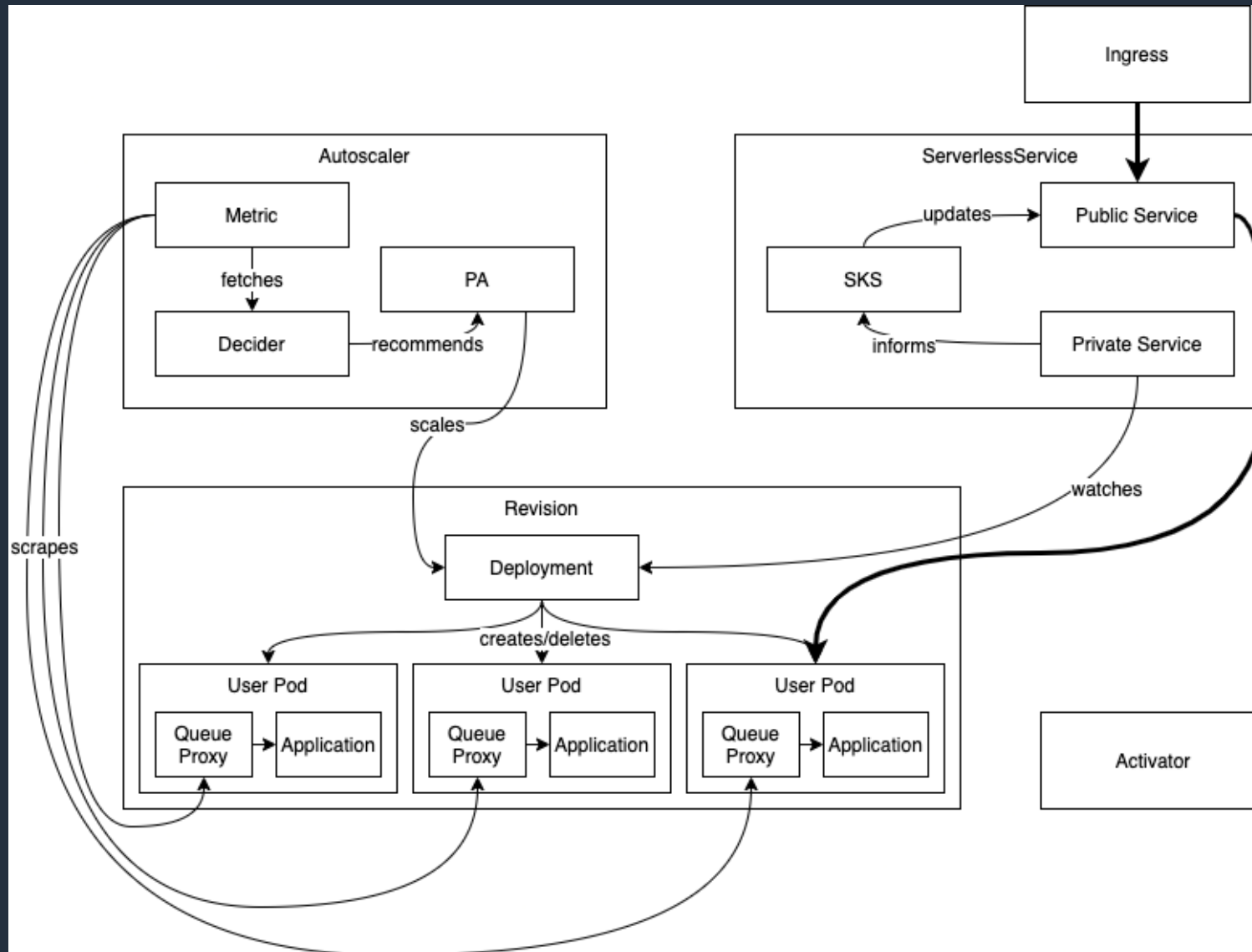
Autoscaling with Knative Pod Autoscaler (KPA)



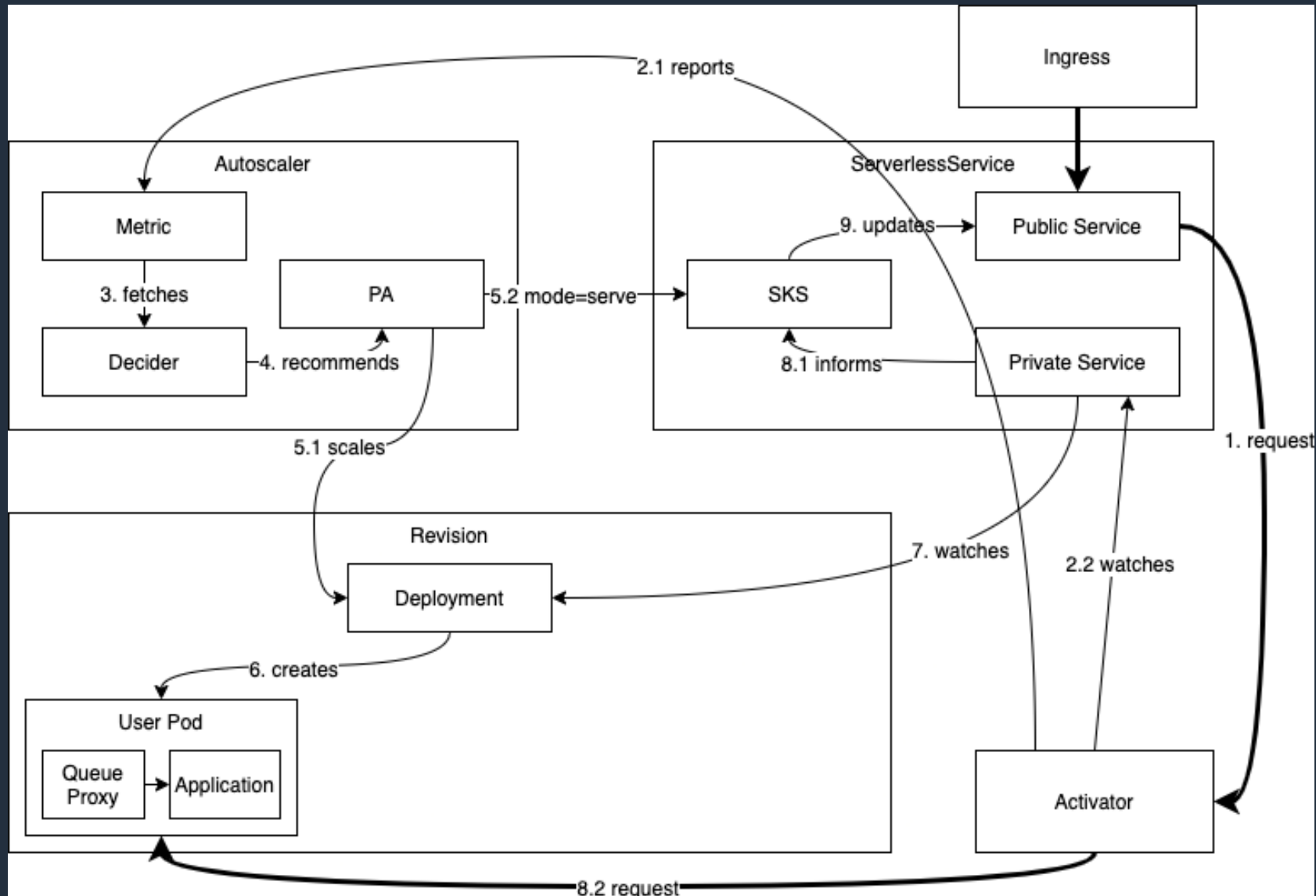
Scaling up and down (steady state)



Scaling to zero



Scaling from Zero



Autoscale Sample



```
apiVersion: serving.knative.dev/v1alpha1
kind: Service
metadata:
  name: autoscale-go
  namespace: default
spec:
  runLatest:
    configuration:
      revisionTemplate:
        spec:
          container:
            image: github.com/knative/docs/
              serving/samples/autoscale-go
```

Ramp up traffic to maintain 10 in-flight requests.



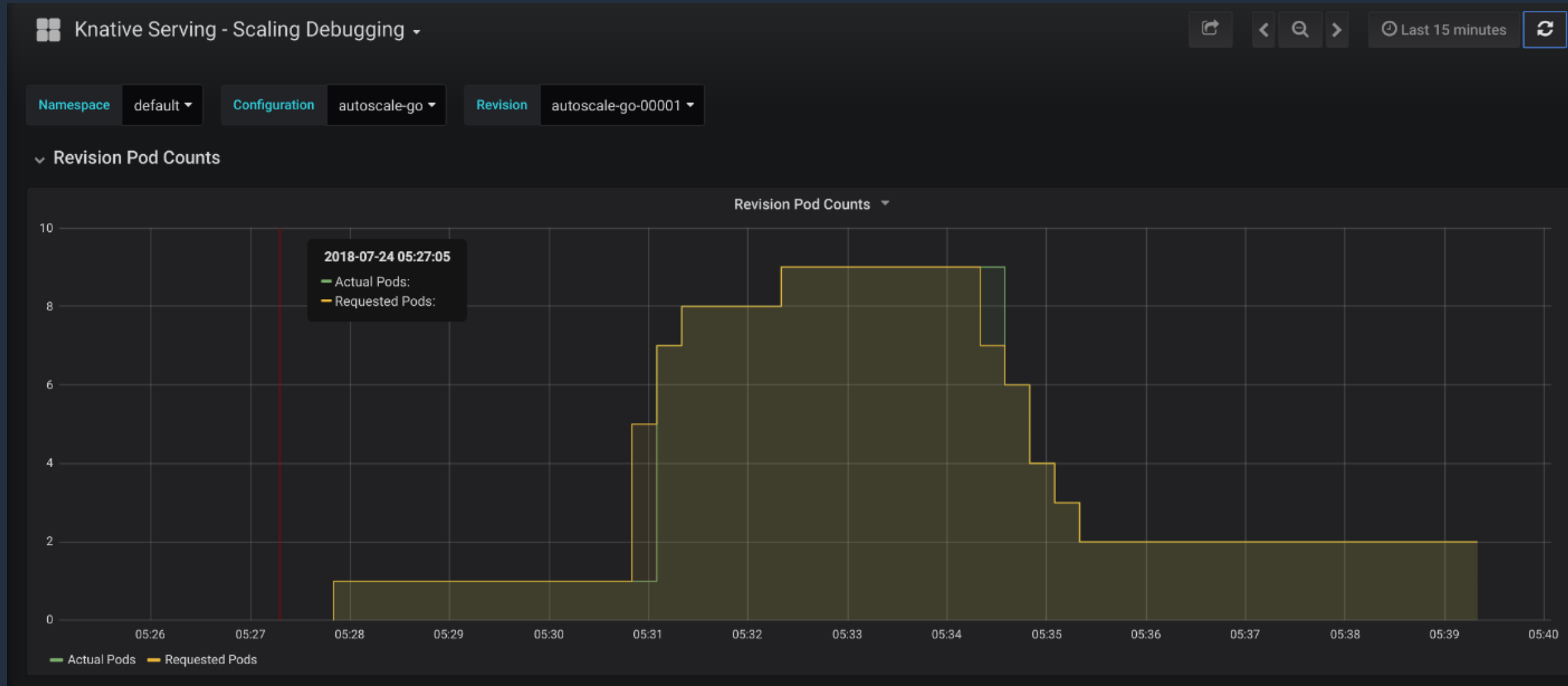
```
go run serving/samples/autoscale-
go/test/test.go \
  -sleep 100 -prime 10000 -bloat 5 -qps 9999
-concurrency 300
```



REQUEST STATS:

Total: 439	Inflight: 299	Done: 439	...	Avg Latency: 0.4655 sec
Total: 1151	Inflight: 245	Done: 712	...	Avg Latency: 0.4178 sec
Total: 1706	Inflight: 300	Done: 555	...	Avg Latency: 0.4794 sec
Total: 2334	Inflight: 264	Done: 628	...	Avg Latency: 0.5207 sec
Total: 2911	Inflight: 300	Done: 577	...	Avg Latency: 0.4401 sec
...				

Scaling pod from zero



Difference between KPA and HPA

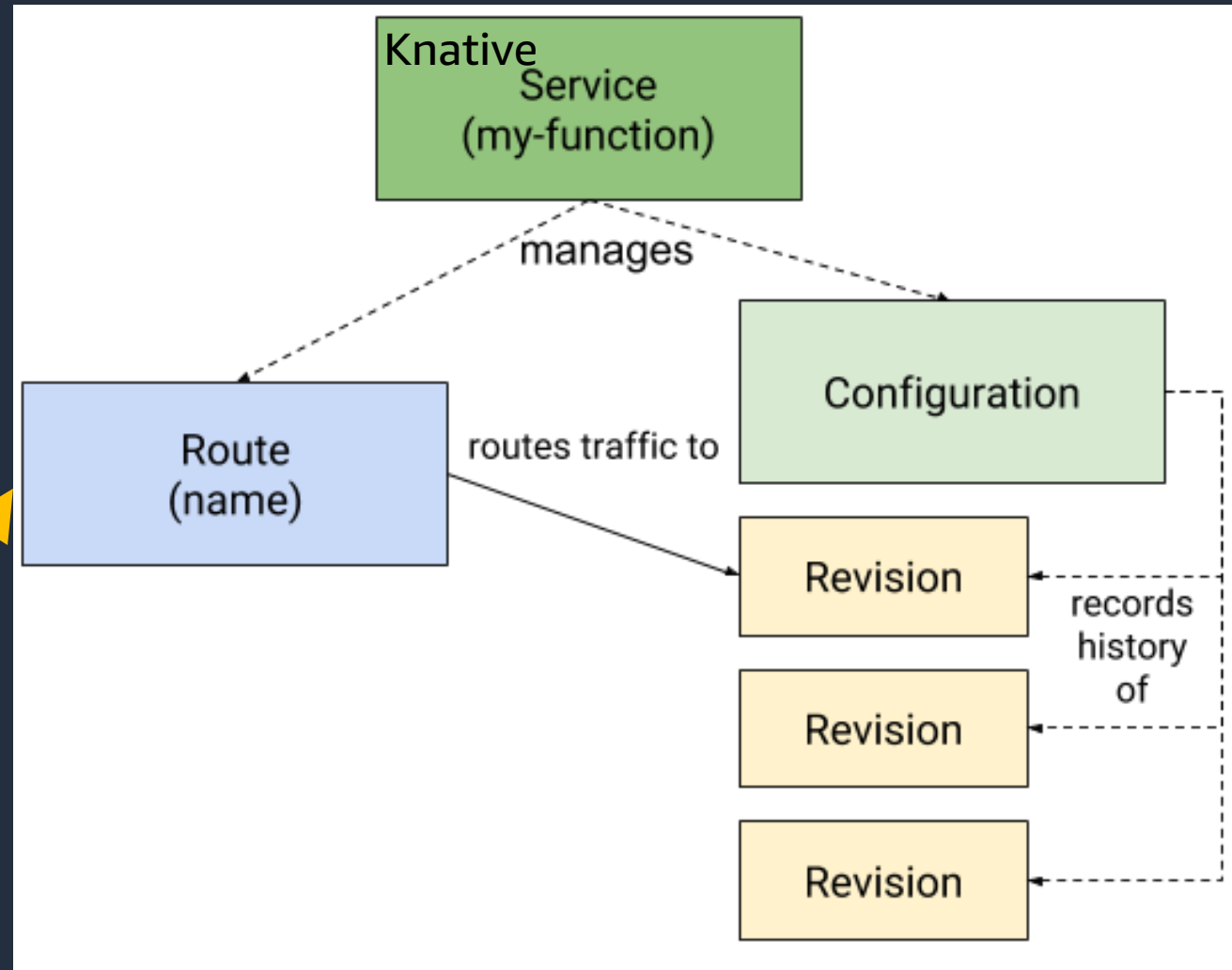
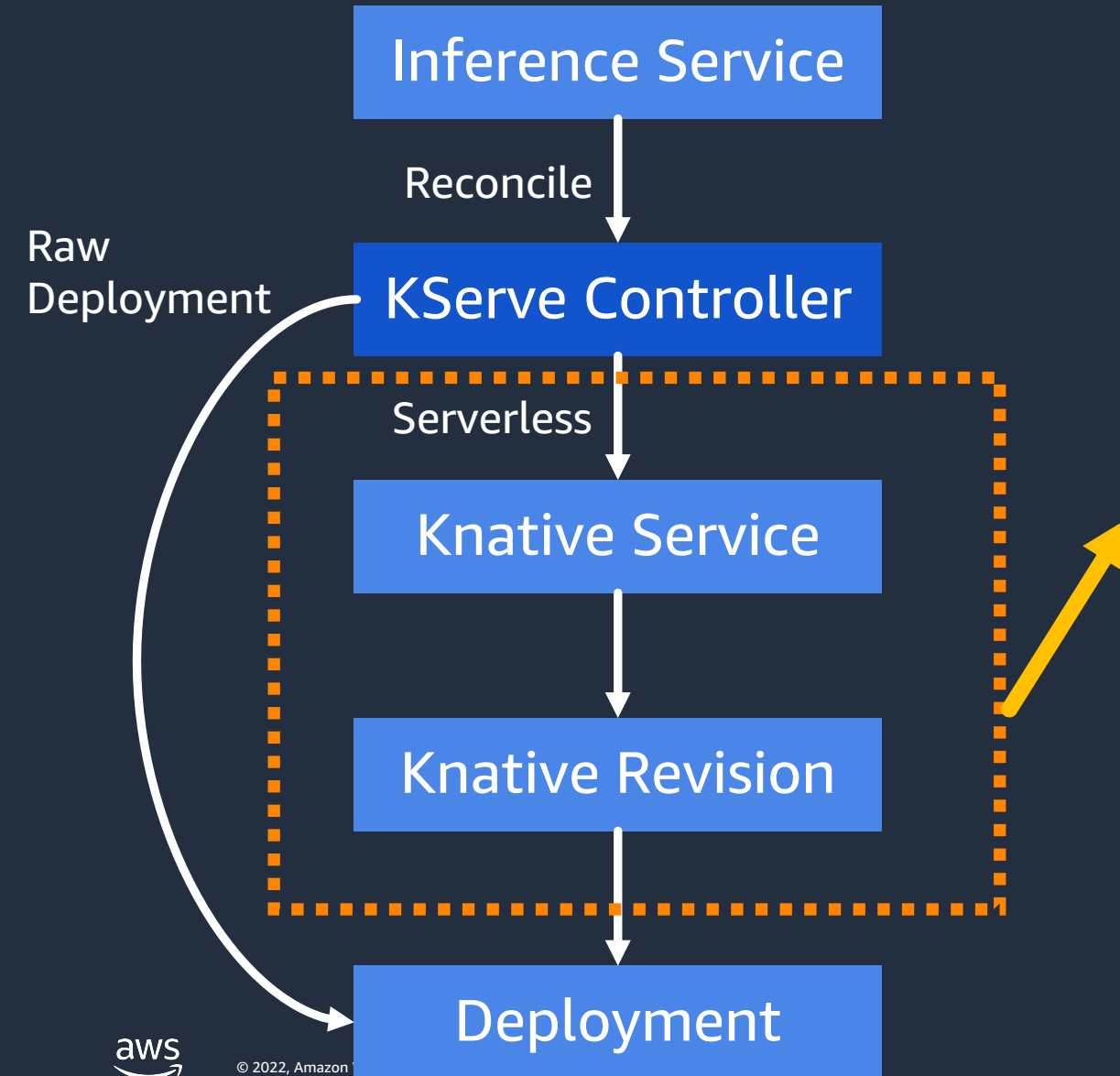
Knative Pod Autoscaler (KPA)

- Part of the Knative Serving core and enabled by default once Knative Serving is installed.
- Supports scale to zero functionality.
- Does not support CPU-based autoscaling.

Horizontal Pod Autoscaler (HPA)

- Not part of the Knative Serving core, and must be enabled after Knative Serving installation.
- Does not support scale to zero functionality.
- Supports CPU-based autoscaling.

We have covered Knative Serving part...

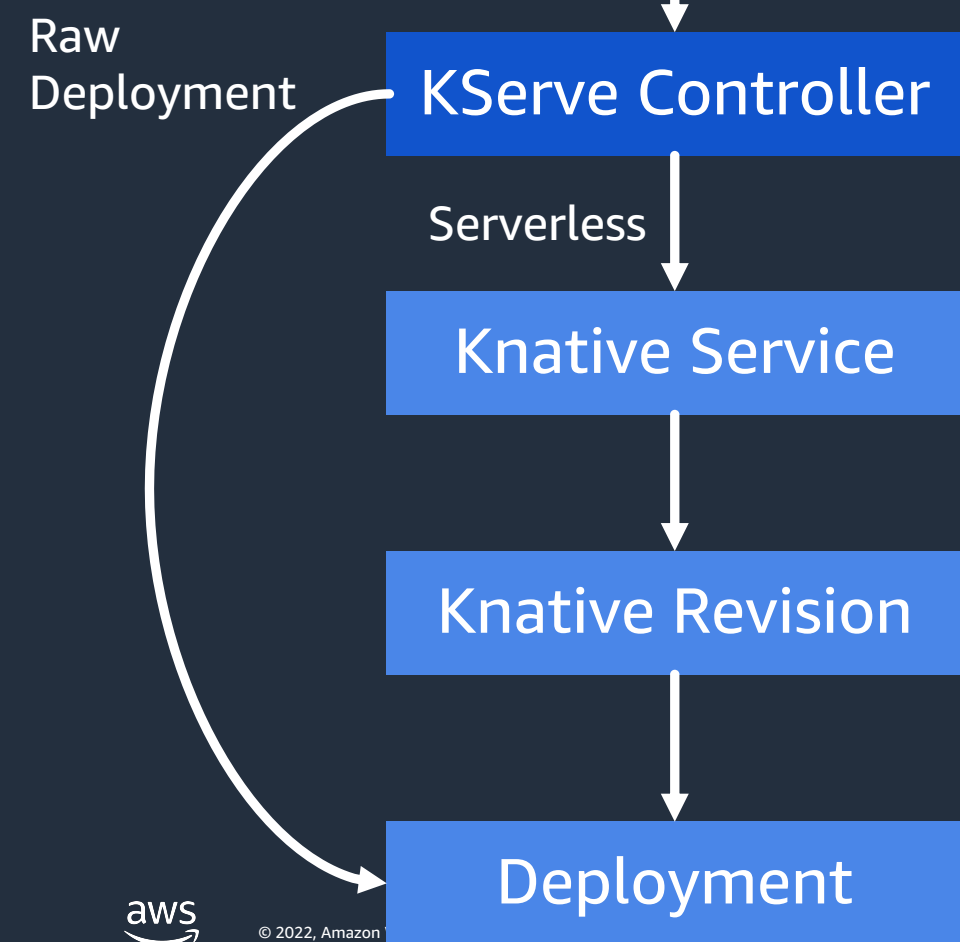


Up next: Inference Service



Question: Do we have to deal with the complexity in Knative?

Answer: No! All we need is Inference Service.



```
apiVersion: "serving.kserve.io/v1beta1"
kind: "InferenceService"
metadata:
  name: "sklearn-iris"
spec:
  predictor:
    model:
      modelFormat:
        name: sklearn
      storageUri: "gs://kfserving-
examples/models/sklearn/1.0/model"
```


First InferenceService



```
apiVersion:
"serving.kserve.io/v1beta1"
kind: "InferenceService"
metadata:
  name: "sklearn-iris"
spec:
  predictor:
    model:
      modelFormat:
        name: sklearn
      storageUri: "gs://kfserving-
examples/models/sklearn/1.0/model"
```

Apply



```
InferenceService/sklearn-iris
├── Service/sklearn-iris
├── Service/sklearn-iris-predictor-default
│   ├── Configuration/sklearn-iris-predictor-default
│   │   └── Revision/sklearn-iris-predictor-default-00001
│   │       ├── Deployment/sklearn-iris-predictor-default-00001-deployment
│   │       │   ├── ReplicaSet/sklearn-iris-predictor-default-00001-deployment-77f7cbb544
│   │       │   │   └── Pod/sklearn-iris-predictor-default-00001-deployment-77f7cbb54497gd8
│   │       ├── Image/sklearn-iris-predictor-default-00001-cache-kserve-container
│   │       └── PodAutoscaler/sklearn-iris-predictor-default-00001
│   │           ├── Metric/sklearn-iris-predictor-default-00001
│   │           └── ServerlessService/sklearn-iris-predictor-default-00001
│   │               ├── Endpoints/sklearn-iris-predictor-default-00001
│   │               │   └── EndpointSlice/sklearn-iris-predictor-default-00001-98kpg
│   │               ├── Service/sklearn-iris-predictor-default-00001
│   │               └── Service/sklearn-iris-predictor-default-00001-private
│   │                   └── EndpointSlice/sklearn-iris-predictor-default-00001-private-xg6r4
│   └── Route/sklearn-iris-predictor-default
│       ├── Ingress/sklearn-iris-predictor-default
│       │   ├── VirtualService/sklearn-iris-predictor-default-ingress
│       │   └── VirtualService/sklearn-iris-predictor-default-mesh
│       └── Service/sklearn-iris-predictor-default
└── VirtualService/sklearn-iris
```

First Inference Service

```
cat <<EOF > "./iris-input.json"
{
  "instances": [
    [6.8, 2.8, 4.8, 1.4],
    [6.0, 3.4, 4.5, 1.6]
  ]
}
EOF
```

```
SERVICE_HOSTNAME=$(kubectl get inferencservice
sklearn-iris -n kserve-test -o
jsonpath='{.status.url}' | cut -d "/" -f 3)
```

```
curl -v -H "Host: ${SERVICE_HOSTNAME}"
"http://${INGRESS_HOST}:${INGRESS_PORT}/v1/models
/sklearn-iris:predict" -d @./iris-input.json
```

```
* Trying 127.0.0.1:8081...
* Connected to localhost (127.0.0.1) port 8081
(#0)
> POST /v1/models/sklearn-iris:predict HTTP/1.1
> Host: sklearn-iris.kserve-test.example.com
...
<
* Connection #0 to host localhost left intact
{"predictions":[1,1]}
```

First Inference Service: load test



```
# use kubectl create instead of apply because the job template  
# is using generateName which doesn't work with kubectl apply  
kubectl create -f https://raw.githubusercontent.com/  
kserve/kserve/release-0.8/docs/samples/v1beta1/sklearn/v1/perf.yaml -n kserve-test
```

Under the hood

NAME	READY	STATUS	RESTARTS	AGE
load-test95k5g-2qktp	0/1	Completed	0	102s
sklearn-iris-predictor-default-00001-deployment-77f7cbb54497gd8	2/2	Running	0	38m

Requests	[total, rate, throughput]	30000, 500.02, 499.55
Duration	[total, attack, wait]	1m0s, 59.998s, 56.662ms
Latencies	[min, mean, 50, 90, 95, 99, max]	3.581ms, 49.293ms, 49.148ms, 57.565ms, 62.676ms, 85.408ms, 153.4ms
Bytes In	[total, mean]	630000, 21.00
Bytes Out	[total, mean]	2460000, 82.00
Success	[ratio]	100.00%
Status Codes	[code:count]	200:30000
Error Set:		



Thank you!